

Splitting code

[RecursiveCharacterTextSplitter] (https://reference.langchain.com/python/langchain_text_splitters/#langchain_text_splitters.RecursiveCharacterTextSplitter) includes pre-built lists of separators that are useful for [splitting text] (/oss/python/integrations/splitters/) in a specific programming language.

Supported languages are stored in the ``langchain_text_splitters.Language`` enum. They include:

```

...
"cpp",
"go",
"java",
"kotlin",
"js",
"ts",
"php",
"proto",
"python",
"rst",
"ruby",
"rust",
"scala",
"swift",
"markdown",
"latex",
"html",
"sol",
"csharp",
"cobol",
"c",
"lua",
"perl",
"haskell"
...

```

To view the list of separators for a given language, pass a value from this enum into

```

```python theme={null}
RecursiveCharacterTextSplitter.get_separators_for_language
```

```

To instantiate a splitter that is tailored for a specific language, pass a value from the enum into

```

```python theme={null}
RecursiveCharacterTextSplitter.from_language
```

```

Below we demonstrate examples for the various languages.

```

```python theme={null}
pip install -qU langchain-text-splitters
```

```

```

```python theme={null}
from langchain_text_splitters import (
 Language,
 RecursiveCharacterTextSplitter,
)
```

```

To view the full list of supported languages:

```

```python theme={null}

```

```
[e.value for e in Language]
```

```
```
```

```
```output theme={null}
```

```
['cpp',
 'go',
 'java',
 'kotlin',
 'js',
 'ts',
 'php',
 'proto',
 'python',
 'rst',
 'ruby',
 'rust',
 'scala',
 'swift',
 'markdown',
 'latex',
 'html',
 'sol',
 'csharp',
 'cobol',
 'c',
 'lua',
 'perl',
 'haskell',
 'elixir',
 'powershell',
 'visualbasic6']
```

```
```
```

You can also see the separators used for a given language:

```
```python theme={null}
```

```
RecursiveCharacterTextSplitter.get_separators_for_language(Language.PYTHON)
```

```
```
```

```
```output theme={null}
```

```
['\nclass ', '\ndef ', '\n\tdef ', '\n\n', '\n', ' ', '']
```

```
```
```

Python

Here's an example using the PythonTextSplitter:

```
```python theme={null}
```

```
PYTHON_CODE = """
def hello_world():
 print("Hello, World!")
```

```
Call the function
```

```
hello_world()
```

```
"""
```

```
python_splitter = RecursiveCharacterTextSplitter.from_language(
 language=Language.PYTHON, chunk_size=50, chunk_overlap=0
)
```

```
python_docs = python_splitter.create_documents([PYTHON_CODE])
```

```
python_docs
```

```
```
```

```
```output theme={null}
```

```
[Document(metadata={}, page_content='def hello_world():\n print("Hello, World!")'),
 Document(metadata={}, page_content='# Call the function\nhello_world()')]
```

```
```
```

JS

Here's an example using the JS text splitter:

```
```python theme={null}
JS_CODE = """
function helloWorld() {
 console.log("Hello, World!");
}

// Call the function
helloWorld();
"""

js_splitter = RecursiveCharacterTextSplitter.from_language(
 language=Language.JS, chunk_size=60, chunk_overlap=0
)
js_docs = js_splitter.create_documents([JS_CODE])
js_docs
```

```output theme={null}
[Document(metadata={}, page_content='function helloWorld() {\n console.log("Hello, World!");\n}'),
 Document(metadata={}, page_content='// Call the function\nhelloWorld();')]
```
```

TS

Here's an example using the typescript text splitter:

```
```python theme={null}
TS_CODE = """
function helloWorld(): void {
 console.log("Hello, World!");
}

// Call the function
helloWorld();
"""

ts_splitter = RecursiveCharacterTextSplitter.from_language(
 language=Language.TS, chunk_size=60, chunk_overlap=0
)
ts_docs = ts_splitter.create_documents([TS_CODE])
ts_docs
```

```output theme={null}
[Document(metadata={}, page_content='function helloWorld(): void {}'),
 Document(metadata={}, page_content='console.log("Hello, World!");\n'),
 Document(metadata={}, page_content='// Call the function\nhelloWorld();')]
```
```

Markdown

Here's an example using the Markdown text splitter:

```
```python theme={null}
markdown_text = """
🦜🔗 LangChain
```

⚡ Building applications with LLMs through composability ⚡

## ## What is LangChain?

# Hopefully this code block isn't split  
LangChain is a framework for...

As an open-source project in a rapidly developing field, we are extremely open to contributions.

```
"""
...

```python theme={null}
md_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.MARKDOWN, chunk_size=60, chunk_overlap=0
)
md_docs = md_splitter.create_documents([markdown_text])
md_docs
```

```output theme={null}
[Document(metadata={}, page_content='# 🦜🔗 LangChain'),
 Document(metadata={}, page_content='⚡ Building applications with LLMs through composability ⚡'),
 Document(metadata={}, page_content='## What is LangChain?'),
 Document(metadata={}, page_content='# Hopefully this code block isn't split'),
 Document(metadata={}, page_content='LangChain is a framework for...'),
 Document(metadata={}, page_content='As an open-source project in a rapidly developing field, we'),
 Document(metadata={}, page_content='are extremely open to contributions.')]
```
```

## ## Latex

Here's an example on Latex text:

```
```python theme={null}
latex_text = """
\documentclass{article}

\begin{document}

\maketitle

\section{Introduction}
Large language models (LLMs) are a type of machine learning model that can be trained on vast amounts
of text data to generate human-like language. In recent years, LLMs have made significant advances in
a variety of natural language processing tasks, including language translation, text generation, and
sentiment analysis.

\subsection{History of LLMs}
The earliest LLMs were developed in the 1980s and 1990s, but they were limited by the amount of data
that could be processed and the computational power available at the time. In the past decade,
however, advances in hardware and software have made it possible to train LLMs on massive datasets,
leading to significant improvements in performance.

\subsection{Applications of LLMs}
LLMs have many applications in industry, including chatbots, content creation, and virtual
assistants. They can also be used in academia for research in linguistics, psychology, and
computational linguistics.

\end{document}
"""
...

```python theme={null}
latex_splitter = RecursiveCharacterTextSplitter.from_language(
 language=Language.MARKDOWN, chunk_size=60, chunk_overlap=0
)
latex_docs = latex_splitter.create_documents([latex_text])
```

```
latex_docs
'''
```

```
'''output theme={null}
[Document(metadata={}, page_content='\\documentclass{article}\\n\\x08begin{document}\\n\\n\\maketitle'),
 Document(metadata={}, page_content='\\section{Introduction}'),
 Document(metadata={}, page_content='Large language models (LLMs) are a type of machine learning'),
 Document(metadata={}, page_content='model that can be trained on vast amounts of text data to'),
 Document(metadata={}, page_content='generate human-like language. In recent years, LLMs have'),
 Document(metadata={}, page_content='made significant advances in a variety of natural language'),
 Document(metadata={}, page_content='processing tasks, including language translation, text'),
 Document(metadata={}, page_content='generation, and sentiment analysis.'),
 Document(metadata={}, page_content='\\subsection{History of LLMs}'),
 Document(metadata={}, page_content='The earliest LLMs were developed in the 1980s and 1990s,'),
 Document(metadata={}, page_content='but they were limited by the amount of data that could be'),
 Document(metadata={}, page_content='processed and the computational power available at the'),
 Document(metadata={}, page_content='time. In the past decade, however, advances in hardware and'),
 Document(metadata={}, page_content='software have made it possible to train LLMs on massive'),
 Document(metadata={}, page_content='datasets, leading to significant improvements in'),
 Document(metadata={}, page_content='performance.'),
 Document(metadata={}, page_content='\\subsection{Applications of LLMs}'),
 Document(metadata={}, page_content='LLMs have many applications in industry, including'),
 Document(metadata={}, page_content='chatbots, content creation, and virtual assistants. They'),
 Document(metadata={}, page_content='can also be used in academia for research in linguistics,'),
 Document(metadata={}, page_content='psychology, and computational linguistics.'),
 Document(metadata={}, page_content='\\end{document}')]
'''
```

## ## HTML

Here's an example using an HTML text splitter:

```
'''python theme={null}
html_text = """
<!DOCTYPE html>
<html>
 <head>
 <title>🦜🔗 LangChain</title>
 <style>
 body {
 font-family: Arial, sans-serif;
 }
 h1 {
 color: darkblue;
 }
 </style>
 </head>
 <body>
 <div>
 <h1>🦜🔗 LangChain</h1>
 <p>⚡ Building applications with LLMs through composability ⚡ </p>
 </div>
 <div>
 As an open-source project in a rapidly developing field, we are extremely open to
 contributions.
 </div>
 </body>
</html>
'''

'''python theme={null}
html_splitter = RecursiveCharacterTextSplitter.from_language(
 language=Language.HTML, chunk_size=60, chunk_overlap=0
)
```

```
html_docs = html_splitter.create_documents([html_text])
html_docs

```output theme={null}
[Document(metadata={}, page_content='<!DOCTYPE html>\n<html>'),
 Document(metadata={}, page_content='<head>\n          <title>🦜🔗 LangChain</title>'),
 Document(metadata={}, page_content='<style>\n          body {\n          font-family:
Aria'),
 Document(metadata={}, page_content='l, sans-serif;\n          }\n          h1 {'),
 Document(metadata={}, page_content='color: darkblue;\n          }\n          </style>\n          </head>'),
 Document(metadata={}, page_content='>'),
 Document(metadata={}, page_content='<body>'),
 Document(metadata={}, page_content='<div>\n          <h1>🦜🔗 LangChain</h1>'),
 Document(metadata={}, page_content='<p>⚡ Building applications with LLMs through composability
⚡ '),
 Document(metadata={}, page_content='</p>\n          </div>'),
 Document(metadata={}, page_content='<div>\n          As an open-source project in a rapidly dev'),
 Document(metadata={}, page_content='eloping field, we are extremely open to contributions.'),
 Document(metadata={}, page_content='</div>\n          </body>\n</html>')]
```
```

## ## Solidity

Here's an example using the Solidity text splitter:

```
```python theme={null}
SOL_CODE = """
pragma solidity ^0.8.20;
contract HelloWorld {
    function add(uint a, uint b) pure public returns(uint) {
        return a + b;
    }
}
"""

sol_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.SOL, chunk_size=128, chunk_overlap=0
)
sol_docs = sol_splitter.create_documents([SOL_CODE])
sol_docs

```output theme={null}
[Document(metadata={}, page_content='pragma solidity ^0.8.20;'),
 Document(metadata={}, page_content='contract HelloWorld {\n function add(uint a, uint b) pure
public returns(uint) {\n return a + b;\n }\n}')]
```
```

C\#

Here's an example using the C# text splitter:

```
```python theme={null}
C_CODE = """
using System;
class Program
{
 static void Main()
 {
 int age = 30; // Change the age value as needed

 // Categorize the age without any console output
 if (age < 18)

```

```

 {
 // Age is under 18
 }
 else if (age >= 18 && age < 65)
 {
 // Age is an adult
 }
 else
 {
 // Age is a senior citizen
 }
}
}
"""
c_splitter = RecursiveCharacterTextSplitter.from_language(
 language=Language.CSHARP, chunk_size=128, chunk_overlap=0
)
c_docs = c_splitter.create_documents([C_CODE])
c_docs
```



```

```output theme={null}
[Document(metadata={}, page_content='using System;'),
 Document(metadata={}, page_content='class Program\n{\n    static void Main()\n    {\n        int age
= 30; // Change the age value as needed'),
 Document(metadata={}, page_content='// Categorize the age without any console output\n        if
(age < 18)\n        {\n            // Age is under 18'),
 Document(metadata={}, page_content='}\n        else if (age >= 18 && age < 65)\n        {\n
// Age is an adult\n        }\n        else\n        {'),
 Document(metadata={}, page_content='// Age is a senior citizen\n        }\n    }\n}')]
```

```


```

Haskell

Here's an example using the Haskell text splitter:

```

```python theme={null}
HASKELL_CODE = """
main :: IO ()
main = do
 putStrLn "Hello, World!"
-- Some sample functions
add :: Int -> Int -> Int
add x y = x + y
"""
haskell_splitter = RecursiveCharacterTextSplitter.from_language(
 language=Language.HASKELL, chunk_size=50, chunk_overlap=0
)
haskell_docs = haskell_splitter.create_documents([HASKELL_CODE])
haskell_docs
```



```

```output theme={null}
[Document(metadata={}, page_content='main :: IO ()'),
 Document(metadata={}, page_content='main = do\n    putStrLn "Hello, World!"\n-- Some'),
 Document(metadata={}, page_content='sample functions\nadd :: Int -> Int -> Int\nadd x y'),
 Document(metadata={}, page_content='= x + y')]
```

```


```

PHP

Here's an example using the PHP text splitter:

```

```python theme={null}
PHP_CODE = """<?php

```

```

namespace foo;
class Hello {
 public function __construct() { }
}
function hello() {
 echo "Hello World!";
}
interface Human {
 public function breath();
}
trait Foo { }
enum Color
{
 case Red;
 case Blue;
}"""
php_splitter = RecursiveCharacterTextSplitter.from_language(
 language=Language.PHP, chunk_size=50, chunk_overlap=0
)
php_docs = php_splitter.create_documents([PHP_CODE])
php_docs
```

```python
output theme={null}
[Document(metadata={}, page_content='<?php\nnamespace foo;'),
 Document(metadata={}, page_content='class Hello {'),
 Document(metadata={}, page_content='public function __construct() { }\n)'),
 Document(metadata={}, page_content='function hello() {\n echo "Hello World!";\n)'),
 Document(metadata={}, page_content='interface Human {\n public function breath();\n)'),
 Document(metadata={}, page_content='trait Foo { }\nenum Color\n{\n case Red;'),
 Document(metadata={}, page_content='case Blue;\n}')]
```

```

PowerShell

Here's an example using the PowerShell text splitter:

```

python theme={null}
POWERSHELL_CODE = """
$directoryPath = Get-Location

$items = Get-ChildItem -Path $directoryPath

$files = $items | Where-Object { -not $_.PSIsContainer }

$sortedFiles = $files | Sort-Object LastWriteTime

foreach ($file in $sortedFiles) {
    Write-Output ("Name: " + $file.Name + " | Last Write Time: " + $file.LastWriteTime)
}
"""

powershell_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.POWERSHELL, chunk_size=100, chunk_overlap=0
)
powershell_docs = powershell_splitter.create_documents([POWERSHELL_CODE])
powershell_docs
```

```python
output theme={null}
[Document(metadata={}, page_content='$directoryPath = Get-Location\n\n$items = Get-ChildItem -Path $directoryPath'),
 Document(metadata={}, page_content='$files = $items | Where-Object { -not $_.PSIsContainer }'),
 Document(metadata={}, page_content='$sortedFiles = $files | Sort-Object LastWriteTime'),
 Document(metadata={}, page_content='foreach ($file in $sortedFiles) {'),
 Document(metadata={}, page_content='Write-Output ("Name: " + $file.Name + " | Last Write Time: " +

```

```
$file.LastWriteTime)\n}']
```

```

```
Visual Basic 6
```

```
```python theme={null}
VISUALBASIC6_CODE = """Option Explicit
```

```
Public Sub HelloWorld()
    MsgBox "Hello, World!"
End Sub
```

```
Private Function Add(a As Integer, b As Integer) As Integer
    Add = a + b
End Function
"""
```

```
visualbasic6_splitter = RecursiveCharacterTextSplitter.from_language(
    Language.VISUALBASIC6,
    chunk_size=128,
    chunk_overlap=0,
)
visualbasic6_docs = visualbasic6_splitter.create_documents([VISUALBASIC6_CODE])
visualbasic6_docs
```
```

```
```output theme={null}
[Document(metadata={}, page_content='Option Explicit'),
 Document(metadata={}, page_content='Public Sub HelloWorld()\n    MsgBox "Hello, World!"\nEnd Sub'),
 Document(metadata={}, page_content='Private Function Add(a As Integer, b As Integer) As Integer\nAdd = a + b\nEnd Function')]
```
```

```

```

```
<Callout icon="pen-to-square" iconType="regular">
 [Edit the source of this page on GitHub.](https://github.com/langchain-ai/docs/edit/main/src/oss/integrations/splitters/code_splitter.mdx)
</Callout>
```

```
<Tip icon="terminal" iconType="regular">
 [Connect these docs programmatically](/use-these-docs) to Claude, VSCode, and more via MCP for real-time answers.
</Tip>
```

```

```

```
> To find navigation and other pages in this documentation, fetch the llms.txt file at:
https://docs.langchain.com/llms.txt
```